1) Quick summary (what we'll do)

1. Install Docker and docker-compose plugin on Ubuntu.
2. Create directories for Sharry config / data.
3. Place docker-compose.yml and sharry.conf into the host.
4. Start the service with docker compose up -d.
5. (Optional) Use PostgreSQL and reverse proxy / TLS for production.

Key facts:

- Official Sharry Docker image: eikek0/sharry. Docker Hub
- Default web port is **9090** and web UI is served under /app (so http://HOST:9090/app). eikek.github.io
- Configuration can be provided as a HOCON/JSON file or via environment variables prefixed with SHARRY_. eikek.github.io

---

2) Prerequisites (Ubuntu)

Tested on Ubuntu 20.04+ (commands should work on 22.04 too).

Install Docker & docker-compose plugin:

```
# update and prerequisites
sudo apt update
sudo apt install -y ca-certificates curl gnupg lsb-release

# add Docker official GPG and repo
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" \
  | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin

# optional: allow running docker without sudo (log out/ in or reboot required)
sudo usermod -aG docker $USER
```

(If you prefer docker-compose v1, install separately; modern Docker uses the plugin docker compose.)

---

3) Create directory layout on the host

I recommend /opt/sharry for system installs:

```
sudo mkdir -p /opt/sharry/{data,config,db}
sudo chown -R $USER:$USER /opt/sharry
cd /opt/sharry
```

- data will hold file storage when using filesystem store (or H2 DB files).
- config will contain sharry.conf (HOCON) if you provide it.
- db used if you run a local DB (optional).

---

4) Minimal docker-compose.yml (Sharry with built-in H2 DB)

Create /opt/sharry/docker-compose.yml and paste:

```
version: "3.8"
services:
  sharry:
    image: eikek0/sharry:latest
    container_name: sharry
    restart: unless-stopped
    ports:
      - "9090:9090"
    volumes:
      - ./config/sharry.conf:/opt/sharry.conf:ro        # optional config file (see sample)
      - ./data:/var/lib/sharry/data                     # persistent data (files + H2 db)
    environment:
      # Example env vars (optional). See config docs for names.
      - SHARRY_BIND_ADDRESS=0.0.0.0
      - SHARRY_BIND_PORT=9090
      # - SHARRY_BASE_URL=https://sharry.example.com
    healthcheck:
      test: ["CMD", "curl", "-fsSL", "http://localhost:9090/app"]
      interval: 30s
      timeout: 5s
      retries: 3
```

Notes:

- Image eikek0/sharry is the official Docker image. Docker Hub
- Port mapping 9090:9090 exposes the web UI on host port **9090**. eikek.github.io

Start it:

```
cd /opt/sharry
docker compose up -d
```

Access: http://<your-server-ip>:9090/app

5) Sample sharry.conf (HOCON) — basic admin + base URL + filesystem store

Create /opt/sharry/config/sharry.conf (HOCON). This file is optional — you can use env vars instead — but HOCON gives you richer options.

Example sharry.conf:

```
sharry.restserver {
  base-url = "http://127.0.0.1:9090"    # change to your public URL, e.g. "https://sharry.example.com"
  bind {
    address = "0.0.0.0"
    port = 9090
  }

  backend {
    # Enable a simple admin user for initial login (fixed login module)
    auth {
      fixed {
        enabled = true
        user = "admin"
        password = "ChangeMeNow"   # change immediately after first login
        order = 10
      }
    }

    # Use a filesystem store for files; otherwise default is database (H2)
    share {
      # Example: keep default DB behavior; if you want files on FS:
    }

    # JDBC example (comment out if using builtin H2). If you want postgres:
    # jdbc {
    #   url = "jdbc:postgresql://db:5432/sharrydb"
    #   user = "sharryuser"
    #   password = "sharrypassword"
    # }
  }

  files {
    default-store = "filesystem"
    stores = {
      filesystem = {
        enabled = true
        type = "file-system"
        directory = "/var/lib/sharry/data/files"
        clean-empty-dirs = true
      }
      database = {
        enabled = false
        type = "default-database"
      }
    }
  }

  rest {
    # other restserver options...
  }
}
```

Important:

- The docs explain env var mapping: config options like sharry.restserver.bind.address map to SHARRY_BIND_ADDRESS — env vars take priority. eikek.github.io

If you use the filesystem store, ensure the target inside the container exists (/var/lib/sharry/data/files in example) and is backed by the Docker volume ./data.

---

6) Optional: Docker Compose with PostgreSQL (recommended for production)

Create /opt/sharry/docker-compose.postgres.yml:

```
version: "3.8"
services:
  db:
    image: postgres:15
    restart: unless-stopped
    environment:
      POSTGRES_DB: sharrydb
      POSTGRES_USER: sharryuser
      POSTGRES_PASSWORD: change_me
    volumes:
      - ./db/postgres:/var/lib/postgresql/data

  sharry:
    image: eikek0/sharry:latest
    restart: unless-stopped
    ports:
      - "9090:9090"
    volumes:
      - ./config/sharry.conf:/opt/sharry.conf:ro
      - ./data:/var/lib/sharry/data
    environment:
      - SHARRY_BIND_ADDRESS=0.0.0.0
      - SHARRY_BIND_PORT=9090
      - SHARRY_JDBC_URL=jdbc:postgresql://db:5432/sharrydb
      - SHARRY_JDBC_USER=sharryuser
      - SHARRY_JDBC_PASSWORD=change_me
    depends_on:
      - db
```

Important: The mapping of JDBC config to environment variables may vary — prefer setting the JDBC block inside sharry.conf (HOCON) and mount it. The docs show you can override config via env vars which are derived from config key names. eikek.github.io+1

Bring it up:

```
docker compose -f docker-compose.postgres.yml up -d
```

Then visit http://your-host:9090/app.

---

7) Post-install tips & security

- **Change the admin password immediately** and disable fixed login once you configure OAuth or other auth methods. (See auth config in docs.) eikek.github.io

- For production: put Sharry behind an HTTPS reverse proxy (Nginx, Traefik) and set sharry.restserver.base-url to your public https://... so generated links and cookies work. The docs include a reverse proxy guide. eikek.github.io+1
- Backups: if you use H2 (default), the DB file is in the data folder — back up ./data. If you use Postgres, use pg dumps or volume backups. eikek.github.io
- If you seed the DB with an admin via fixed auth, consider switching to OAuth or other production auth and remove the hardcoded admin. eikek.github.io

---

8) Example commands recap (copy-paste)

```
# (1) create folders
sudo mkdir -p /opt/sharry/{data,config,db}
sudo chown -R $USER:$USER /opt/sharry
cd /opt/sharry

# (2) save the docker-compose.yml shown above into /opt/sharry/docker-compose.yml
# (3) save sharry.conf into /opt/sharry/config/sharry.conf

# (4) start
docker compose up -d

# (5) logs
docker compose logs -f sharry
```

---

9) Where I pulled the info (quick references)

- Official Sharry docs — configuration, environment variable mapping and examples. eikek.github.io+1
- Official Docker image on Docker Hub (eikek0/sharry). Docker Hub
- Official repo docker-compose examples (repo has docker/docker-compose.yml).

Below is a **clear step-by-step guide** on how to change the **Sharry logo** in a **Docker setup**. This applies to any setup running the official Docker image eikek0/sharry.

Sharry allows custom UI assets (logo, CSS, JS…) by mounting your own files into the container.

✅ 1. Make a folder for your custom assets

On your host machine (Ubuntu):

mkdir -p /opt/sharry/custom-assets

Place your logo there, for example:

/opt/sharry/custom-assets/logo.png

☞ PNG is recommended.
☞ Name it whatever you want, e.g. logo.png.

✅ 2. Create a custom CSS file to override the logo

Sharry uses a CSS variable for the logo:

--logo-image: url("/assets/logo.png")

But since we want our own file, we override it.

Create:

nano /opt/sharry/custom-assets/custom.css

Paste this:

```
:root {
  --logo-image: url("/custom/logo.png"); /* This will load your own logo */
}

/* Optional: adjust size */
.logo img {
  width: 180px !important;
}
```

⚠ Note: /custom/logo.png is the path we'll use inside the container later.

✅ 3. Mount your custom assets into the container

Edit your docker-compose.yml.
Add this under the volumes: section of your **sharry** service:

```
volumes:
  - ./config/sharry.conf:/opt/sharry.conf:ro
  - ./data:/var/lib/sharry/data
  - /opt/sharry/custom-assets:/opt/sharry/custom:ro
```

Full example:

```
services:
  sharry:
    image: eikek0/sharry:latest
    container_name: sharry
    restart: unless-stopped
    ports:
      - "9090:9090"
    volumes:
      - ./config/sharry.conf:/opt/sharry.conf:ro
      - ./data:/var/lib/sharry/data
      - /opt/sharry/custom-assets:/opt/sharry/custom:ro
```

✅ 4. Tell Sharry to use your custom CSS & assets

Edit your Sharry config (sharry.conf):

nano /opt/sharry/config/sharry.conf

Add this block under sharry.restserver:

```
sharry.restserver {
  # ...your other settings...

  ui {
```

```
  custom-assets = [
   {
    path = "/opt/sharry/custom/custom.css"
    mount-path = "/assets/custom.css"
   },
   {
    path = "/opt/sharry/custom/logo.png"
    mount-path = "/custom/logo.png"
   }
  ]

  # Load your CSS into the UI
  css-files = [
   "/assets/custom.css"
  ]
 }
}
```

Explanation:

- path → where the file is on the host (mounted into container)
- mount-path → where Sharry exposes it inside the web UI

So Sharry will read your custom CSS and your logo.

✅ 5. Restart Sharry

```
docker compose down
docker compose up -d
```

Wait a few seconds, then open:

http://YOUR-IP:9090/app

Hard refresh the browser:

- **Ctrl + F5**
- or clear cache

Your new logo should now appear.